# CLASSIFICATION OF HAND-WRITTEN DIGITS

ÁLVARO GALISTEO BERMÚDEZ, DANIEL RATH

## Contents



*Date*: February 10, 2026.

## 1. Logistic regression for multi-class classification

Logistic regression is a commonly used model for classification tasks. In our case we want to use logistic regression for multi-class classification, which is a generalization of the binary logistic regression.

In our case, we want to predict the class of a given $8 \times 8$ grayscale image of a hand written digit. One would expect that a given input can not be classified as more than one digit at the same time. Therefore, we want to compute the probability/score of a given input image to belong to every of the 10 classes (digits from 0 to 9) and assign the class with the highest probability/score to the input image.

### 1.1. **Our problem.**

An input image is represented as a vector $a_j \in \mathbb{R}^{64}$. Each entry of the vector is an integer between 0 and 16 and corresponds to the intensity of a pixel in the image with 0 being the lowest intensity and 16 being the highest intensity. Each image $a_j$ in the dataset is associated with a label $y_j \in \{0, \dots, 9\}$, which is the digit that is represented in the image. In order to use the labels in our model, we encode them into one-hot vectors $p_j \in \{0, 1\}^{10}$, where the $i$-th entry is 1 if the label is $i$ and 0 otherwise.[Dev]

We want to train a function $\varphi : \mathbb{R}^{64} \times (\mathbb{R}^{64} \times \mathbb{R})^{10} \to \mathbb{R}^{10}$ that takes an input image $a_j$ and a parameter $x = (w_i, b_i)_{i=0}^{9}$ and outputs a vector of scores for each class. The score for class $i$ is computed as $\varphi(a_j; x)_i = a_j^T w_i + b_i$. The $w_i \in \mathbb{R}^{64}$ is a weight vector and $b_i \in \mathbb{R}$ is a bias term for class $i$.

The training of the model consists of the following optimization problem

$$\underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \sum_{j=1}^{N} L(p_j, \varphi(a_j; x)),$$

with $N \in \mathbb{N}$ being the number of training samples and $L$ the cross entropy loss defined as

$$L(p, s) = \log\Big( \sum_{l=0}^{9} \exp(s_l) \Big) - \sum_{i=0}^{9} p_i \, s_i. \tag{1.1}$$

After the model has been trained, we can use it to classify new input images $a$ by computing $\varphi(a; x)$ and assigning the class with the highest score to the input image, i.e.

$$\tilde{y} = \underset{i \in \{0, \dots, 9\}}{\arg\max} \, \varphi(a; x)_i.$$

The values of $\varphi(a_j; x)_i$ can be interpreted as probabilities of the input image $a_j$ belonging to class $i$ after applying the softmax function, i.e.

$$\sigma(s)_i = \frac{\exp(s_i)}{\sum_{l=0}^{9} \exp(s_l)}.$$

### 1.2. **Solvability.**

The given optimization problem is a convex problem. This is because the log-sum-exp function is convex as we have already proved in the lecture. For reference one can also check [Boy09, pp. 72, 74]. Also the second term of our loss function (1.1) is linear and therefore convex. Also, $\varphi$ is an affine mapping in $x$. And at the same time convex function over an affine function is also convex (cf. [Boy09, p. 67]).

In general however, the optimization problem is not necessarily solvable. This is because the loss function (1.1) is not strictly convex in its scores $s$ and therefore not strictly convex in the parameter $x$.

This, is because the loss function (1.1) is invariant under the addition of the same constant to every score $s_i$. More precisely, for a constant $c \in \mathbb{R}$ we have

$$\log\Big(\sum_{l=0}^{9} \exp\big(s_l + c\big)\Big) = \log\Big(\exp(c)\sum_{l=0}^{9} \exp\big(s_l\big)\Big) = \log\Big(\sum_{l=0}^{9} \exp\big(s_l\big)\Big) + c$$

and also

$$\sum_{i=0}^{9} p_i(s_i + c) = \sum_{i=0}^{9} p_i s_i + c.$$

Therefore, we have

$$L(p, s + c\mathbb{1}) = \log\Big(\sum_{l=0}^{9} \exp\big(s_l + c\big)\Big) - \sum_{i=0}^{9} p_i(s_i + c) = L(p, s) + c - c = L(p, s).$$

This implies that for a solution $x$ of our optimization problem, we get arbitrary additional solutions, e.g. $x' = (w_i + c\mathbb{1}, b_i)_{i=0}^{9}$ for any constant $c \in \mathbb{R}$. Therefore, the optimization problem is not uniquely solvable.

Even if we ignore non-uniqueness, the minimum in general may fail to be attained at all. A reason is linear separability of the training data. For a parameter $x$ and

$$s_{j,i} := \varphi(a_j; x)_i = a_j^\top w_i + b_i,$$

and $p_{j,y_j} = 1$, we can rewrite the loss as

$$L(p_j, s_j) = \log\Big(\sum_{l=0}^{9} e^{s_{j,l}}\Big) - s_{j,y_j} = \log\Big(1 + \sum_{i \neq y_j} e^{-(s_{j,y_j} - s_{j,i})}\Big) \geq 0. \tag{1.2}$$

Hence $L(p_j, s_j)$ is gets small when all the $s_{j,y_j} - s_{j,i}$ are large, e.g. when the correct class score $s_{j,y_j}$ dominates all other scores $s_{j,i}$ by a large margin $\gamma > 0$. If there exists a parameter $\bar{x} = (\bar{w}_i, \bar{b}_i)_{i=0}^{9}$ such that this is the case, we call the dataset *separable*.

We are then able to scale the parameter $\bar{x}$ with a factor $t > 0$

$$\bar{x}^{(t)} := (t\bar{w}_i, t\bar{b}_i)_{i=0}^{9}.$$

and get $s_{j,i}^{(t)} = \varphi(a_j; x^{(t)})_i = t\, s_{j,i}$. Thus, for the differences we then get

$$s_{j,y_j}^{(t)} - s_{j,i}^{(t)} = t(s_{j,y_j} - s_{j,i}) \geq t\gamma.$$

If we use this, the above expression (1.2) yields

$$0 \leq L\big(p_j, \varphi(a_j; \bar{x}^{(t)})\big) = \log\Big(1 + \sum_{i \neq y_j} e^{-t(s_{j,y_j} - s_{j,i})}\Big) \leq \log\big(1 + 9e^{-t\gamma}\big) \xrightarrow[t\to\infty]{} 0.$$

Therefore, if there exists such a separating parameter $\bar{x}$, the optimization term $\sum_{j=1}^{N} L(p_j, \varphi(a_j; \bar{x}^{(t)}))$ is going to be arbitrarily small for large $t$, i.e. for large weights and biases $t\bar{w}_i$ and $t\bar{b}_i$. This implies that the minimum of our optimization problem is not attained at a finite parameter $x$ and therefore the optimization problem is not solvable. Explain what is happening in the seperable case from a geometric point of view!

*Remark* 1.2.1. The idea, that linear separability can be a problem for logistic regression, was inspired by [Ada18, Linear Separability and Regularization].

### 1.3. **Regularization.**

solves some problems explain why regularization is a good thing to do!

It is fairly obvious that the optimization problem is $C^\infty$. Where should we put this?

## 2. Stochastic gradient descent

In order to solve the optimization problem of our logistic regression model, we use stochastic gradient descent.

## References


[Ada18] Ryan Adams. *Linear Classification with Logistic Regression. Lecture notes.* COS 324 – Elements of Machine Learning. Princeton University. Oct. 8, 2018. URL: https://www.cs.princeton.edu/courses/archive/spring19/cos324/files/logistic-regression.pdf (visited on 02/10/2026).

[Boy09] Stephen P. Boyd. *Convex optimization.* Ed. by Lieven Vandenberghe. 7th ed. first published 2004. Cambridge University Press, 2009. 716 pp. ISBN: 9780521833783.

[Dev] scikit-learn Developers. *Documentation of scikit-learn Machine Learning in Python.* Version 1.8.0 (stable). URL: https://scikit-learn.org (visited on 02/09/2026).